

System Requirements and Design Specifications

QUBO-Based Sensor Node Placement for Device-Free Localization

Khalil Brik

Spring 2026

Contents

1	Introduction	3
2	System Overview	3
2.1	Purpose	3
2.2	Scope	3
2.3	Stakeholders	3
3	Functional Requirements	3
3.1	FR-1: Data Ingestion	3
3.2	FR-2: QUBO Formulation	4
3.3	FR-3: Binary Search over Alpha	4
3.4	FR-4: Optimization via Annealing Solvers	4
3.5	FR-5: Greedy Baseline	4
3.6	FR-6: Localization Evaluation	5
3.7	FR-7: Time-to-Solution Metric	5
3.8	FR-8: Multi-Run Statistical Experiment	5
3.9	FR-9: Result Persistence	5
3.10	FR-10: Figure Generation	5
4	Non-Functional Requirements	6
4.1	NFR-1: Reproducibility	6
4.2	NFR-2: Performance	6
4.3	NFR-3: Portability	6
4.4	NFR-4: Modularity	6
4.5	NFR-5: Output Quality	6
5	System Architecture	6
5.1	Component Overview	6
5.2	Data Flow	7
5.3	Script Responsibilities	7
6	Key Design Decisions	8

6.1	QUBO Parameterization	8
6.2	Binary Search Instead of Constraint Encoding	8
6.3	OpenJij as the Solver Backend	8
6.4	k-NN as the Localization Evaluator	8
7	System Parameters Summary	8
8	File and Directory Structure	8
9	Dependencies	10
10	How to Run the System	10

Introduction

This document describes the system requirements and design specifications for the QUBO-based sensor node placement framework developed as part of the senior thesis project. The system addresses the problem of selecting which Access Point (AP) and Monitoring Point (MP) pairs to activate in a Device-Free Localization (DFL) environment, given fixed hardware budgets.

The core idea is to formulate the pair selection problem as a Quadratic Unconstrained Binary Optimization (QUBO) problem and solve it using both Simulated Annealing (SA) and Simulated Quantum Annealing (SQA) via the OpenJij library. A greedy baseline is also implemented for comparison. Performance is measured using mean localization error from a k-Nearest Neighbor (k-NN) classifier evaluated on held-out test fingerprints.

System Overview

Purpose

The purpose of the system is to identify the best subset of AP-MP sensor links for indoor fingerprint-based localization. Activating all available links is not always practical due to hardware, power, or cost constraints. The system finds a budget-feasible subset that maximizes localization accuracy while minimizing redundancy between selected links.

Scope

The system covers the full pipeline from raw fingerprint data to evaluated node placement results. This includes data preprocessing, QUBO formulation, optimization via annealing solvers, localization evaluation, and figure generation for the thesis. It does not include real-time hardware deployment or wireless communication protocols.

Stakeholders

- **Thesis author:** develops, runs, and evaluates the system.
- **Thesis committee:** reviews the methodology and results.
- **Future researchers:** may extend the system to real hardware or larger-scale environments.

Functional Requirements

FR-1: Data Ingestion

The system shall load the following precomputed input data from CSV files:

- RSS fingerprint matrices for training and test sets, of shape $K_{\text{train}} \times L$ and $K_{\text{test}} \times L$ respectively, where K is the number of fingerprint locations and L is the total number of candidate AP-MP pairs.
- 2D coordinate matrices for training and test fingerprint locations, each of shape $K \times 2$.

- A pairs table listing the AP index and MP index for each of the L candidate links.
- An importance score vector of length L , where each entry represents the RSS variance of the corresponding link.
- A redundancy matrix of shape $L \times L$, where each entry represents the correlation between the RSS signals of two links.

FR-2: QUBO Formulation

The system shall construct a QUBO matrix Q parameterized by a scalar $\alpha \in [0, 1]$ as follows:

$$Q_{pp} = -\alpha \cdot I_p, \quad p = 1, \dots, L \quad (1)$$

$$Q_{pq} = 2(1 - \alpha) \cdot R_{pq}, \quad p < q \quad (2)$$

where I_p is the importance score of pair p and R_{pq} is the redundancy between pairs p and q . The diagonal terms reward selecting important links, and the off-diagonal terms penalize selecting redundant ones.

FR-3: Binary Search over Alpha

The system shall find the optimal trade-off parameter α^* using binary search over the interval $[0, 1]$ with tolerance $\epsilon = 0.01$. At each iteration, the QUBO is solved and the resulting selection is checked for feasibility. A selection is feasible if the number of distinct AP nodes is at most $n = 5$ and the number of distinct MP nodes is at most $m = 5$. The binary search moves the lower bound up when feasible and the upper bound down when over budget.

FR-4: Optimization via Annealing Solvers

The system shall support two annealing-based solvers:

- **Simulated Annealing (SA):** implemented via `openjij.SASampler` with 100 reads and 1000 sweeps per call.
- **Simulated Quantum Annealing (SQA):** implemented via `openjij.SQASampler` with 10 reads and 1000 sweeps per call, emulating the behavior of a D-Wave quantum annealer with a projected hardware anneal time of $20 \mu\text{s}$ per read.

Both solvers use a fixed random seed of 42 for reproducibility.

FR-5: Greedy Baseline

The system shall implement a deterministic greedy baseline that selects AP-MP pairs in descending order of importance score while enforcing the same $n = 5$ and $m = 5$ budget constraints. This baseline serves as a performance lower bound for comparison against the QUBO-based methods.

FR-6: Localization Evaluation

The system shall evaluate any candidate selection using a k-Nearest Neighbor (k-NN) localizer with $k = 3$. For each test fingerprint, the localizer computes Euclidean distances to all training fingerprints using only the selected subset of RSS measurements, identifies the three nearest neighbors, and predicts the location as their mean coordinate. The evaluation metric is the mean Euclidean error in meters over all test locations.

FR-7: Time-to-Solution Metric

The system shall compute the Time-to-Solution (TTS) for each solver at the optimal α^* . TTS is defined as the expected total annealing time needed to find the optimal solution with probability $p_R = 0.99$:

$$\text{TTS}(\tau, p_R) = \tau \cdot \frac{\ln(1 - p_R)}{\ln(1 - p_s)} \quad (3)$$

where τ is the anneal time per single run and p_s is the empirical success probability estimated from the reads. For SA, τ is the measured wall-clock time per read. For SQA, τ is the projected D-Wave hardware anneal time of 20 μs .

FR-8: Multi-Run Statistical Experiment

The system shall repeat the full binary search pipeline 100 times with different random seeds to produce mean, standard deviation, minimum, and maximum statistics for each solver. The greedy baseline is deterministic and is therefore run once and replicated across all rows for consistency.

FR-9: Result Persistence

The system shall save all outputs to the `data/output/` directory, including the binary search log, selected pair lists for each solver, a per-run multi-run results table, a statistical summary, and a greedy result file.

FR-10: Figure Generation

The system shall generate the following publication-quality figures at 300 DPI and save them to the `figures/` directory:

- Mean localization accuracy vs. α for SA and SQA.
- Time-to-Solution vs. α for both solvers.
- Number of selected pairs vs. α .
- Number of active APs and MPs vs. α .
- Per-solver AP and MP counts at α^* .
- Total computation time per solver.
- Empirical CDF of mean localization error across 100 runs.
- Coverage comparison illustrating clustered, parallel, and diverse link configurations.

- DFL deployment diagram.
- System architecture diagram.

Non-Functional Requirements ---

NFR-1: Reproducibility

All stochastic components of the system shall use a fixed seed so that results can be reproduced exactly from the saved data and scripts.

NFR-2: Performance

The full binary search for a single solver shall complete within a reasonable time on a standard laptop (under 10 minutes for SA with the given parameter settings). The multi-run experiment of 100 trials may run for longer and is intended to be executed once as a batch job.

NFR-3: Portability

The system shall run on any machine with Python 3.8 or later and the following packages installed: `numpy`, `pandas`, `matplotlib`, and `openjij`. No proprietary quantum hardware is required since the quantum annealer is simulated by OpenJij.

NFR-4: Modularity

Each stage of the pipeline is implemented as a separate numbered script. This allows individual stages to be re-run independently without reprocessing the entire pipeline from scratch.

NFR-5: Output Quality

All figures shall be saved at 300 DPI in PNG format (and in PDF format where applicable) using IEEE-style font settings (serif, 9 pt) to ensure they are suitable for direct inclusion in the thesis document.

System Architecture ---

Component Overview

The system is organized into four logical layers, each corresponding to a subdirectory in the repository.

1. **Data layer** (`data/input/`): holds the raw and precomputed CSV files that serve as inputs to the optimization and evaluation pipeline.
2. **Computation layer** (`scripts/`): contains the Python scripts that perform QUBO construction, binary search, evaluation, and baseline comparison.
3. **Output layer** (`data/output/`): stores all numerical results produced by the computation layer.

4. **Visualization layer (figures/)**: stores all figures generated from the output data.

Data Flow

Figure 1 summarizes the data flow through the system.

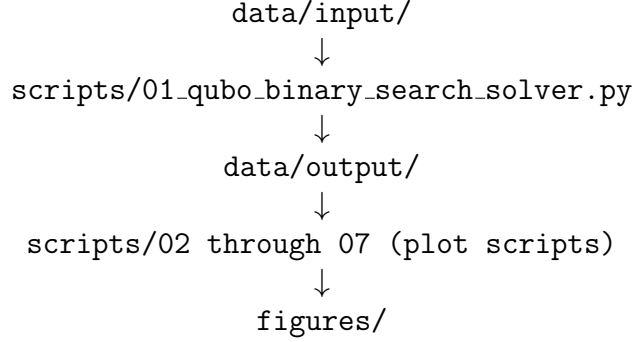


Figure 1: High-level data flow through the system pipeline.

Script Responsibilities

Script	Responsibility
<code>01_qubo_binary_search_solver.py</code>	Builds the QUBO matrix, runs binary search for SA and SQA, evaluates via k-NN, computes TTS, and saves all results.
<code>02_plot_result_figures.py</code>	Reads the binary search log and summary to produce the main result figures (accuracy, TTS, pair counts, solver comparison).
<code>03_greedy_baseline.py</code>	Implements and evaluates the greedy baseline and prints a three-way comparison against SA and SQA.
<code>04_multirun_experiment.py</code>	Repeats the binary search 100 times with varying seeds and saves per-run and summary statistics.
<code>05_plot_coverage_comparison.py</code>	Generates a three-panel figure illustrating clustered, parallel, and diverse link configurations with Fresnel zone overlays.
<code>06_plot_dfl_deployment.py</code>	Generates the DFL deployment diagram showing AP and MP nodes in the environment.
<code>07_plot_system_diagram.py</code>	Generates the system architecture diagram for the thesis.

Table 1: Script responsibilities in the computation layer.

Key Design Decisions

QUBO Parameterization

The single scalar α was chosen as the only tunable parameter of the QUBO formulation. When $\alpha = 1$, the objective reduces to pure importance maximization with no redundancy penalty. When $\alpha = 0$, it becomes pure redundancy minimization. Binary search over α is an efficient way to find the point where budget feasibility is just barely satisfied, which tends to correspond to the most accurate selection.

Binary Search Instead of Constraint Encoding

Budget constraints are enforced through binary search rather than being encoded as penalty terms in the QUBO. This avoids the need to tune penalty weights, which is a common difficulty with constrained QUBO formulations. At each binary search iteration, feasibility is simply checked by counting the distinct AP and MP nodes in the best sample returned by the solver.

OpenJij as the Solver Backend

OpenJij was selected because it provides both SA and SQA samplers through a unified interface that closely mirrors the D-Wave Ocean SDK. This makes it straightforward to switch to real quantum hardware in future work without changing the surrounding pipeline.

k-NN as the Localization Evaluator

A 3-NN classifier was used to evaluate localization quality because it is parameter-light, interpretable, and consistent with prior work in fingerprint-based indoor localization. The same evaluator is used for all solvers so that comparisons are fair.

System Parameters Summary

File and Directory Structure

```
Final_Submission/
  README.md
  system_requirements_and_design.tex      <- this document
  notebooks/
    01_precompute_fingerprint_data.ipynb
    02_qubo_solver_exploration.ipynb
  scripts/
    01_qubo_binary_search_solver.py
    02_plot_result_figures.py
    03_greedy_baseline.py
    04_multirun_experiment.py
    05_plot_coverage_comparison.py
    06_plot_dfl_deployment.py
    07_plot_system_diagram.py
```


Parameter	Value	Description
N	10	Total number of sensor nodes
n (AP budget)	5	Maximum distinct AP nodes allowed
m (MP budget)	5	Maximum distinct MP nodes allowed
ϵ	0.01	Binary search convergence tolerance
k	3	Number of neighbors in k-NN localizer
p_R	0.99	Target success probability for TTS
SA reads	100	Number of reads per QUBO call (SA)
SA sweeps	1000	Number of sweeps per read (SA)
SQA reads	10	Number of reads per QUBO call (SQA)
SQA sweeps	1000	Number of sweeps per read (SQA)
τ_{QA}	20 μs	Projected D-Wave hardware anneal time
Seed	42	Random seed for reproducibility
N_{runs}	100	Number of trials in the multi-run experiment

Table 2: Summary of key system parameters.

```

latex/
  algorithm.tex
  qubo_formulation.tex
data/
  input/
    fingerprint_rss_train.csv
    fingerprint_rss_test.csv
    fingerprint_coords_train.csv
    fingerprint_coords_test.csv
    ap_mp_pairs.csv
    importance_scores.csv
    redundancy_matrix.csv
  output/
    binary_search_log.csv
    selected_pairs_sa.csv
    selected_pairs_qa.csv
    greedy_result.csv
    multirun_results.csv
    multirun_summary.csv
    summary.csv
figures/
  fig00_dfl_deployment.png
  fig01_accuracy.png / .pdf
  fig02_time_to_solution.png / .pdf
  fig03_pairs_vs_alpha.png / .pdf
  fig04_ap_mp_vs_alpha.png / .pdf

```

```

fig05_ap_mp_per_solver.png
fig06_total_computation_time.png
fig07_empirical_cdf.png
fig08_coverage_comparison.png
fig09_system_architecture_diagram.png
coverage_animation.html

```

Dependencies

Package	Version	Purpose
Python	3.8+	Runtime environment
NumPy	any	Numerical arrays and linear algebra
pandas	any	CSV I/O and data manipulation
matplotlib	any	Figure generation
openjij	any	SA and SQA QUBO samplers

Table 3: Software dependencies.

Install all dependencies with:

```
pip install numpy pandas matplotlib openjij
```

How to Run the System

All scripts should be executed from the `Final_Submission/` root directory so that the relative paths `data/` and `figures/` resolve correctly.

1. Run `notebooks/01_precompute_fingerprint_data.ipynb` in Jupyter to generate the input CSV files. Skip this step if the `data/input/` files are already present.
2. Run `scripts/01_qubo_binary_search_solver.py` to perform the binary search for SA and SQA and save the results.
3. Run `scripts/02_plot_result_figures.py` to generate the main result figures.
4. Run `scripts/03_greedy_baseline.py` to evaluate the greedy baseline and compare it against SA and SQA.
5. Run `scripts/04_multirun_experiment.py` to run the 100-trial statistical experiment.
6. Run scripts 05 through 07 in any order to generate the remaining figures.